



L2：流程 & 中斷

吳俊峰

助理教授，國家楊明喬董大學

✉ cfwu417@cs.nycu.edu.tw

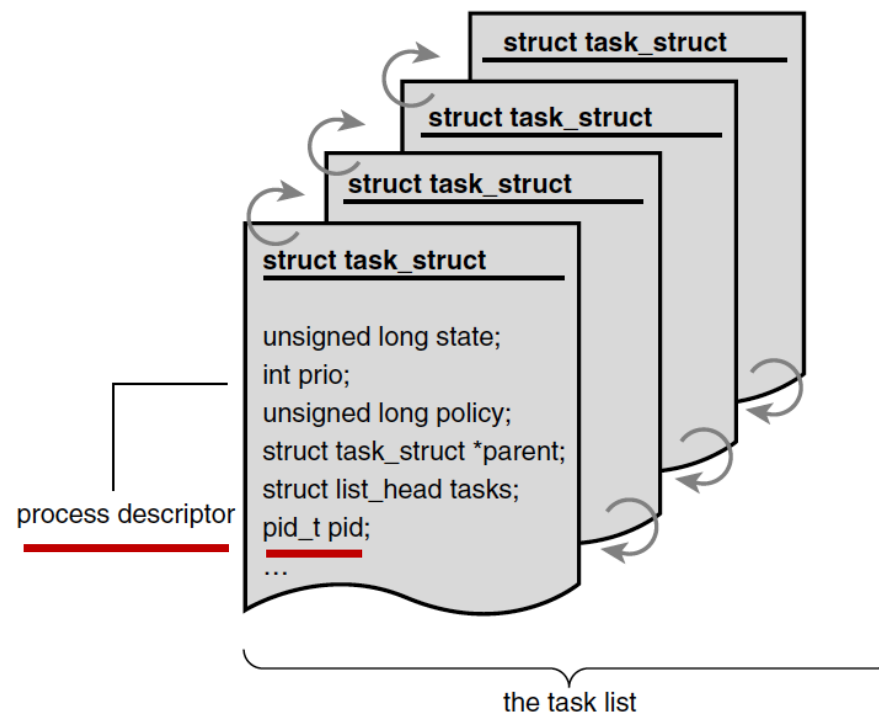
🌐 <https://cfwu417.github.io>



什麼是進程

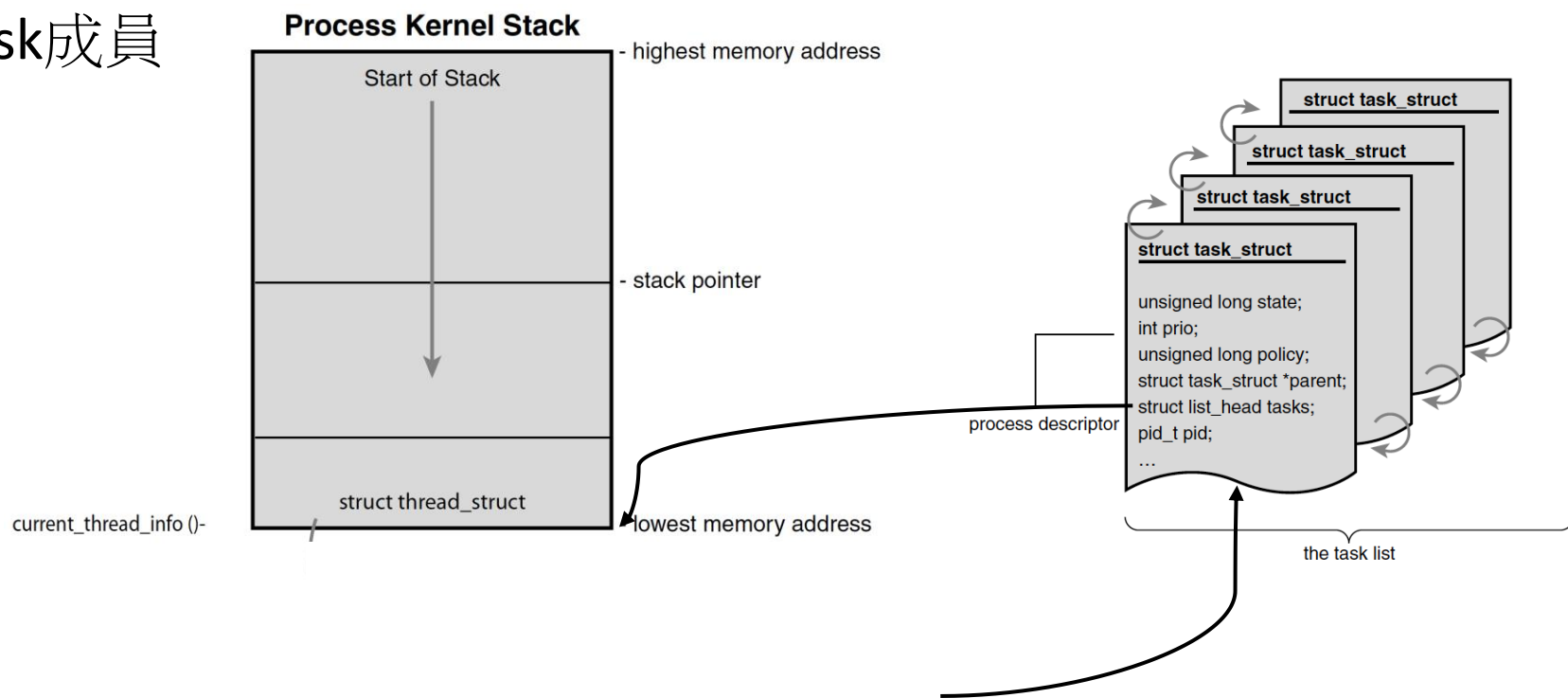
任務清單和流程描述

- 過程：一個活動的程式和相關資源，例如處理器狀態、記憶體位址空間。
- 流程列表 循環雙向連結清單中的描述符。
- 進程描述符（ task_struct ）：包含有關進程的所有資訊。
 - 進程 ID (pid)：唯一的進程識別值
 - 在Linux中，XXX Descriptor表示維護XXX資訊的資料結構。
 - 進程控制塊（統稱）
進程描述符（在Linux中稱為task_struct）



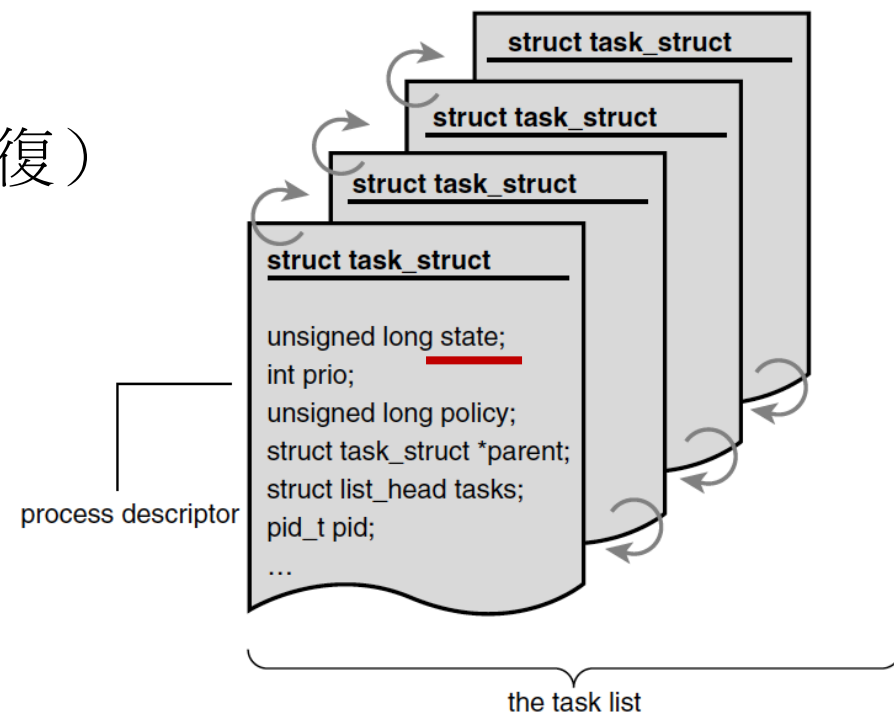
核心如何存取進程描述符

- 核心堆疊：**Linux 核心**使用這個**每個線程的記憶體區域**來儲存核心用來管理線程的中間資訊（例如，返回地址、核心函數呼叫）和元資料塊（`thread_info`）。
- `thread_info`儲存在**核心棧的底部**。（更容易訪問）
 - 核心堆疊大小：靜態2~3頁（4KB）
 - 取消引用`thread_info`的`task`成員獲取**指針**`task_struct`。



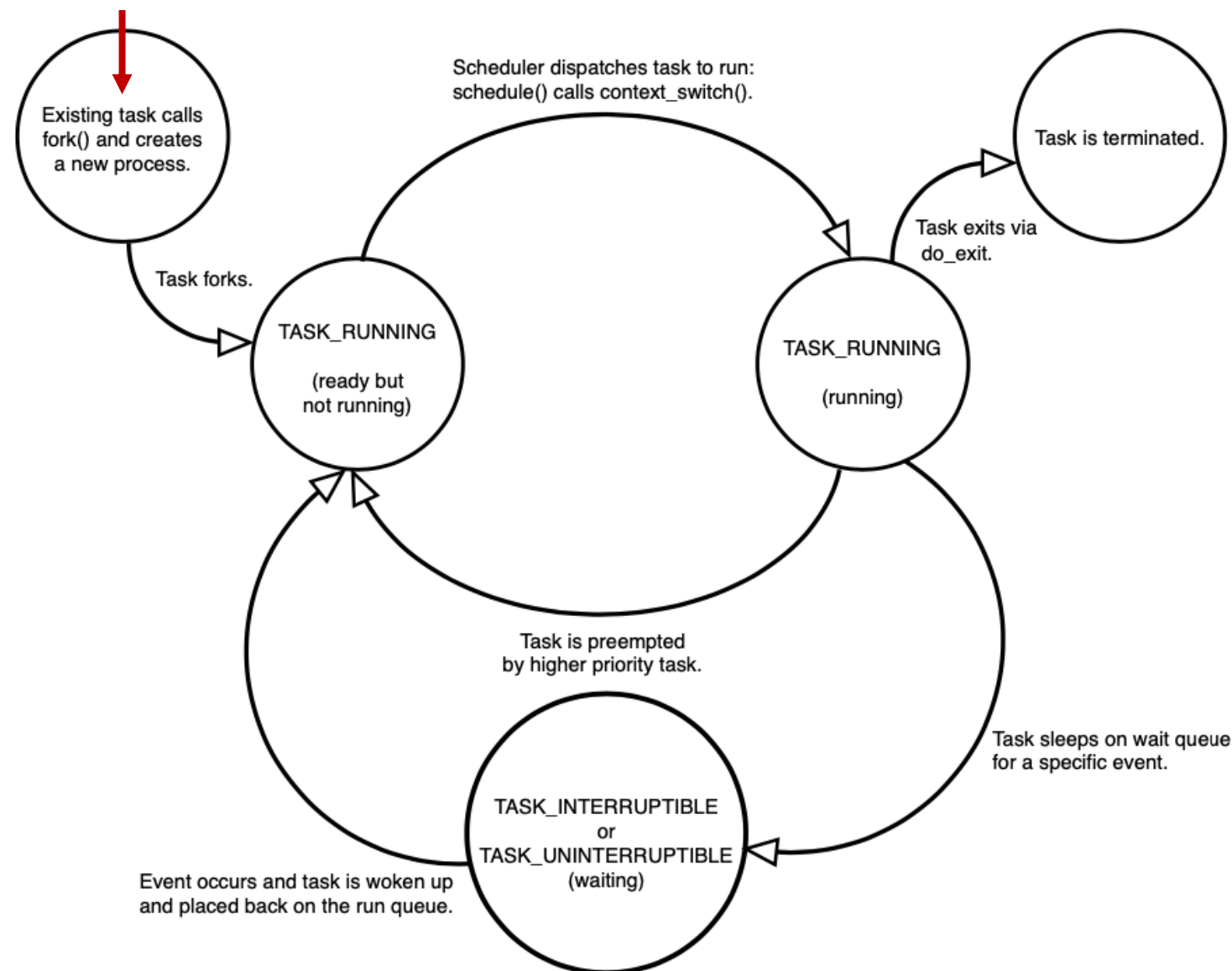
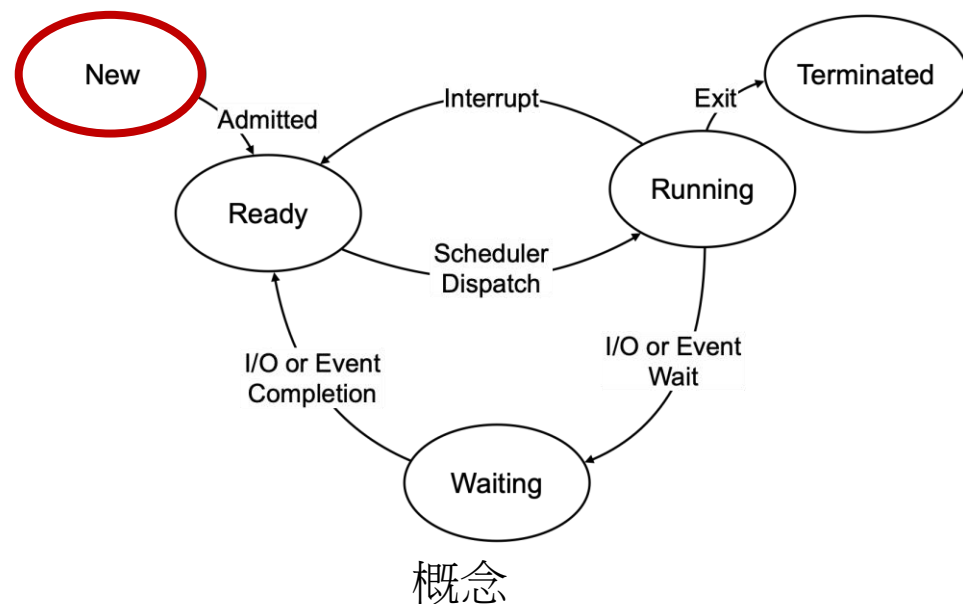
行程狀態

- 進程描述符的**狀態欄位**描述了**進程的當前狀態**（其中之一（**五個州**））
 - Task_Running：**可運行狀態**，表示正在運行或在運行佇列中。（這是進程在使用者空間執行時的唯一狀態）
 - Task_Interruptible：**睡眠**（即阻塞），等待某種條件。
 - Task_Uninterruptible：處於睡眠狀態，即使收到訊號也**無法喚醒**。
 - __Task_Stopped：執行已停止（訊號控制到恢復）
 - __Task_Traced：偵錯器追蹤的進程。



行程狀態圖

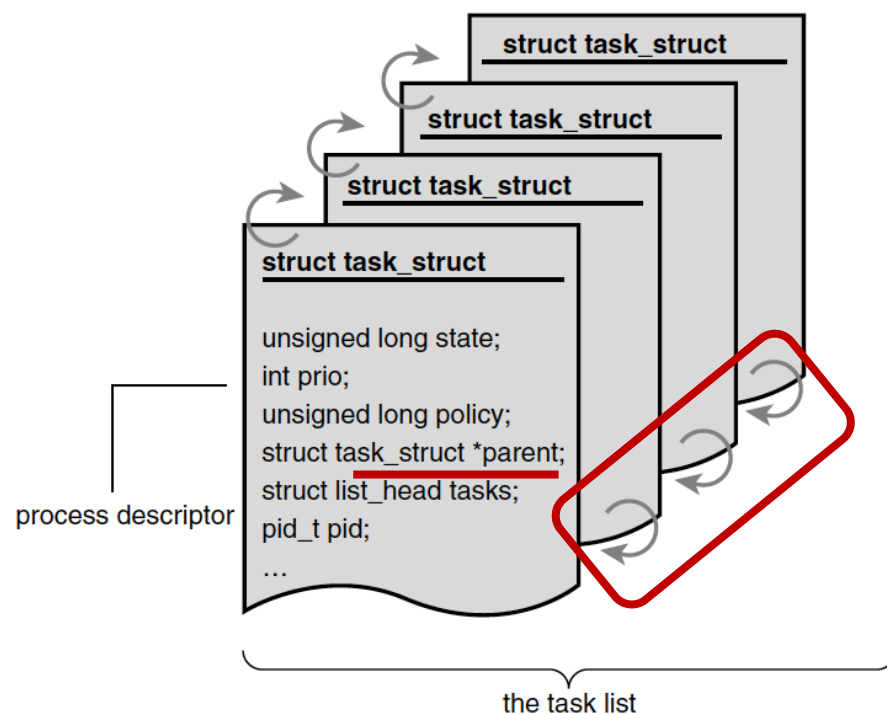
- 生活循環的一個過程。



Linux 核心

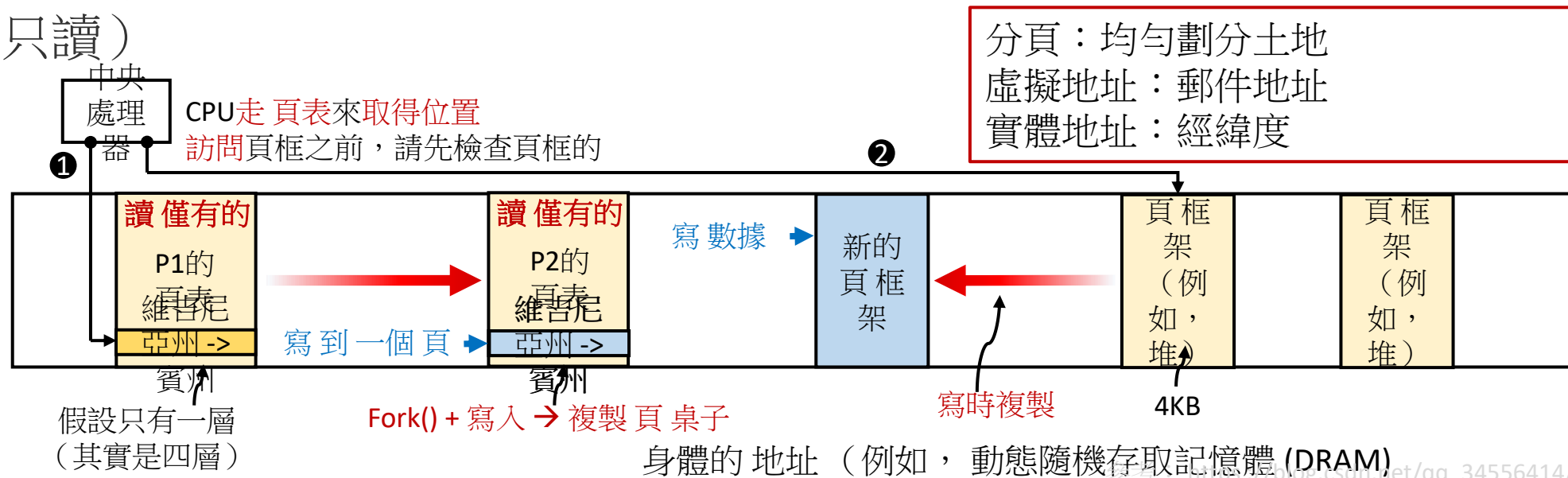
流程家族樹

- 第一個行程：所有行程都是`init`行程（PID 1）的後代
- 如何存取自己的父進程或子進程：分別使用進程描述符（即`task_struct`）提供的父進程或子進程指標。
- 如何存取其他進程：透過遍歷`task_list`。



流程創建

- 過程建立 = `fork()` + `exec()`
 - 叉（）：創造一個孩子經過複製它是父母。（原因：高效率、簡單！）
（註：*PID*，頁表格、頁面框架和一些訊號是不同的）
 - 執行（）：負載一個新的執行檔進入這地址空間（即記憶體）和運行它。
- 寫時複製 (乳牛)
 - 挑戰：天真地複製全部資源從一個父母過程是時間消耗。
 - 如何乳牛作品：延遲和防止複製數據直到這數據是寫的。（IE，資源是共享只讀）



進程創建（計數）

- 過程 建立 = **fork()** + **exec()**
 - 叉（）：創造一個孩子經過複製它是父級。
（註：*PID*，頁桌子，和一些訊號是不同的）
 - 執行()：負載一個新的執行檔進入這地址空間（即記憶體）和運行它。
- 寫時複製 (乳牛)
 - 挑戰：天真地複製全部資源從一個父母過程是時間消耗。
 - 如何乳牛作品：延遲和防止複製數據直到這數據是書面。（IE，資源是共享只讀）
- Linux實作： **copy_process ()**
 - 分配一個新的核心堆疊（例如， **thread_info** ）和程序描述符（即**task_struct** ）
 - 複製（例如，程式計數器、狀態）或設定（例如， **PID**）字段在這過程描述符
 - 放這狀態場地到 **TASK_UNINTERRUPTABLE**（原因：初始化不可中斷）
 - 重複父級的頁表
 - 複製（過程）或者分享（線）：開啟檔案、檔案系統資訊、訊號處理程序、進程位址空間（註：做不是分享堆），和命名空間。

行程與執行緒

- 執行緒（**輕量級進程**）：執行之內這相同的程式在共享記憶地址空間（也分享打開文件和頁表）。
 - Linux 工具全部執行緒作為**標準流程**（有自己的任務結構體 (task_struct)）。
 - 執行緒**有**他們的自己的：（1）程式櫃檯，（2）暫存器，（3）堆，（4）狀態
 - Linux **做不是**提供任何特別的調度或者數據結構到代表線程。
- 創建線程：**複製()**是**完全相同的到又()**，除了那這地址空間，文件系統資源，文件描述符，和訊號處理程式**是共享**。
 - 複製 (CLONE_VM) | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0)
- 核心執行緒：在後台運行核心任務。
 - 與正常進程一樣，**可調度且可搶佔**
 - 但是，與正常進程相比，核心執行緒**沒有位址空間**。
 - **僅在核心空間中操作**，並且不切換到用戶空間。

行程終止

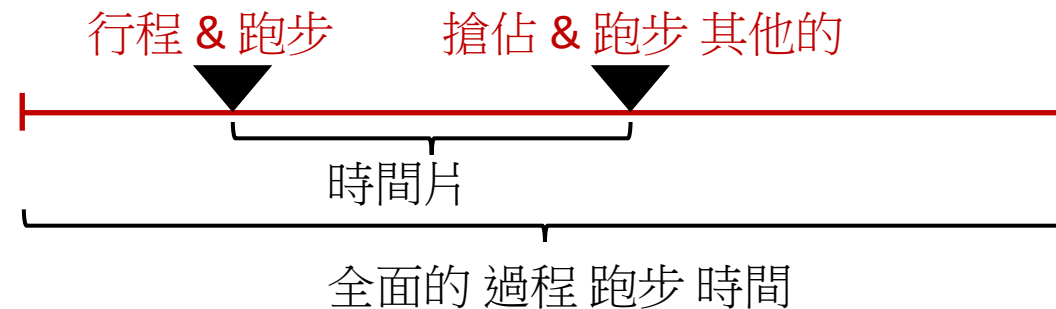
- 當進程或 C 編譯器呼叫 **exit()** 時發生終止。
- **task_struct** 的 **flags** 成員中設定 **PF_EXITING** 標誌。
- 呼叫 **exit_mm()** 釋放本程序佔用的 **mm_struct**（位址空間）。
- 呼叫 **exit_sem()** 從一個 **IPC** 信號量等待佇列中出隊。
- 呼叫 **exit_files()** 和 **exit_fs()** 分別減少與檔案描述符和檔案系統資料相關的物件的**使用計數**。
- 呼叫 **exit_notify()** 向任務的父級發送訊號，將任務的任何子級**重新設定為其執行緒組中的另一個執行緒或init程序**。
- 呼叫 **schedule()** 切換到新進程。
- **父行程檢索資訊**後，該行程的**核心堆疊**（包括 **task_struct** 和 **thread_info**）將被釋放。



行程調度

多任務系統

- 單處理器：分時 和 它是 **虛擬並行**.（多道程式設計）
- 多處理器：運行在不同的處理器同時。
- 調度程序的標準：公平性、回應時間（互動式）、吞吐量
- 合作社：流程 **自願** 屈服中央處理器。
- 先發制人：**不由自主** 暫停(情境轉變) 一個跑步過程。（Linux）
- 時間片：這時間一個過程運行前它是被搶佔。
 - 防止一些流程壟斷處理器。
- 如何到決定這時間片？哪個過程將是跑步下一個？



注意事項 為了 設計 政策

- 進程優先權：
 - 好的 價值： **更大 價值** 代表存在 **更好 到 其他 的**，所以降低優先順序（正常流程）。
 - 即時 的 值：所有即時進程的**優先權都高於普通進程**。
- 時間片（靜止的 映射）：
 - 更長的時間片 → 更好 資源 利用率。
 - 挑戰 1：觸發器 情境 轉變 頻繁地 如果 全部 流程 是 低優先級。
 - 前任：轉變 每一個 5毫秒 如果 好的 價值 是 19
 - 挑戰 2：對於 nice 值較高（優先順序較低）的進程，**時間片 差異**很大
 - 前任： $T_{-20} = T_{-19} + 20$ ， $T_{19} = T_{18} / 2$ （資源 不平衡）
 - CFS 可以解決這些挑戰，但 O(1) 不能。

Nice Value	Time Slice
-20	800ms (DEF_TIMESLICE * 4 * 40 / 20)
-19	780ms (DEF_TIMESLICE * 4 * 39 / 20)
-18	760ms (DEF_TIMESLICE * 4 * 38 / 20)
-17	740ms (DEF_TIMESLICE * 4 * 37 / 20)
-16	720ms (DEF_TIMESLICE * 4 * 36 / 20)
-15	700ms (DEF_TIMESLICE * 4 * 35 / 20)
-14	680ms (DEF_TIMESLICE * 4 * 34 / 20)
-13	660ms (DEF_TIMESLICE * 4 * 33 / 20)
-12	640ms (DEF_TIMESLICE * 4 * 32 / 20)
-11	620ms (DEF_TIMESLICE * 4 * 31 / 20)
-10	600ms (DEF_TIMESLICE * 4 * 30 / 20)
-9	580ms (DEF_TIMESLICE * 4 * 29 / 20)
-8	560ms (DEF_TIMESLICE * 4 * 28 / 20)
-7	540ms (DEF_TIMESLICE * 4 * 27 / 20)
-6	520ms (DEF_TIMESLICE * 4 * 26 / 20)
-5	500ms (DEF_TIMESLICE * 4 * 25 / 20)
-4	480ms (DEF_TIMESLICE * 4 * 24 / 20)
-3	460ms (DEF_TIMESLICE * 4 * 23 / 20)
-2	440ms (DEF_TIMESLICE * 4 * 22 / 20)
-1	420ms (DEF_TIMESLICE * 4 * 21 / 20)
0	100ms (DEF_TIMESLICE * 1 * 20 / 20)
1	95ms (DEF_TIMESLICE * 1 * 19 / 20)
2	90ms (DEF_TIMESLICE * 1 * 18 / 20)
3	85ms (DEF_TIMESLICE * 1 * 17 / 20)
4	80ms (DEF_TIMESLICE * 1 * 16 / 20)
5	75ms (DEF_TIMESLICE * 1 * 15 / 20)
6	70ms (DEF_TIMESLICE * 1 * 14 / 20)
7	65ms (DEF_TIMESLICE * 1 * 13 / 20)
8	60ms (DEF_TIMESLICE * 1 * 12 / 20)
9	55ms (DEF_TIMESLICE * 1 * 11 / 20)
10	50ms (DEF_TIMESLICE * 1 * 10 / 20)
11	45ms (DEF_TIMESLICE * 1 * 9 / 20)
12	40ms (DEF_TIMESLICE * 1 * 8 / 20)
13	35ms (DEF_TIMESLICE * 1 * 7 / 20)
14	30ms (DEF_TIMESLICE * 1 * 6 / 20)
15	25ms (DEF_TIMESLICE * 1 * 5 / 20)
16	20ms (DEF_TIMESLICE * 1 * 4 / 20)
17	15ms (DEF_TIMESLICE * 1 * 3 / 20)
18	10ms (DEF_TIMESLICE * 1 * 2 / 20)
19	5m (DEF_TIMESLICE * 1 * 1 / 20)

Linux 2.6 優先權與時間片

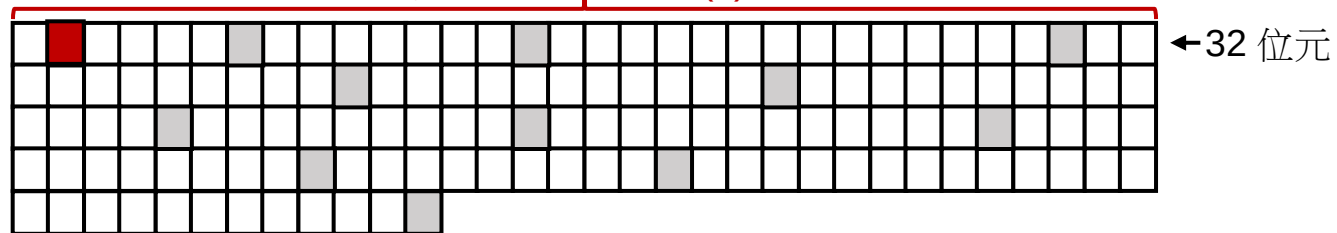
注意事項 為了 設計 政策（數量）

- 過程 優先事項
- 時間片
- 工藝特點：
 - I/O 密集：花費大量時間提交和等待 I/O 請求。
 - 輸入/輸出：鍵盤，網路，貯存
 - 例子：圖形使用者介面（圖形使用者介面）
 - 政策：跑步更多的頻繁互動，更短時間片
 - 處理器受限：花費大量時間執行程式碼。
 - 例子：ssh -keygen，MATLAB
 - 政策：跑步較少的頻繁地，更長時間片
- 維護開銷：
 - 如何找到下一個進程：（1）佇列（2）樹（3）點陣圖
 - 全域運行佇列（鎖定）或每個處理器運行佇列（處理器的使用率）

Linux進程調度器

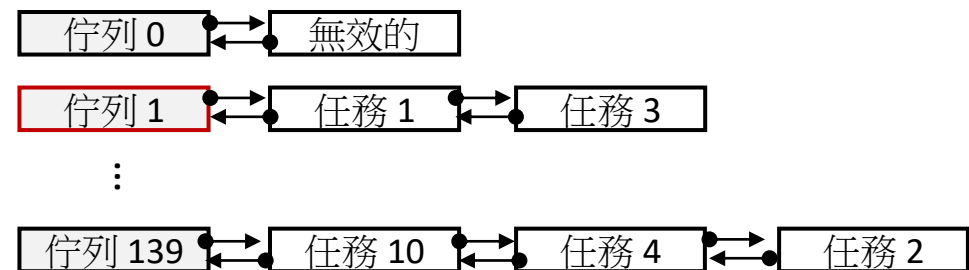
- 在）前 核心 2.4（簡單）：
 - 所有處理器共享一個全域運行佇列。（依優先權對行程進行排序）
 - 不好：當進程用完其時間片時，應鎖定運行佇列。
 - 應依序將進程插入過期佇列。（規模很差什麼時候那裡是噸流程）
- O(1) 核心 2.5（快速）：
 - 查看積極的大批(點陣圖)到取得目標優先級值（最左邊）。
 - 使用這優先事項價值到獲取（出隊）這下一個進程（先進先出）在這積極的大批。
 - 重新計算優先順序並將其排入過期的位數組。
 - 當活動位數組中未設定任何位元時，切換活動位數組和過期位數組。

找出最左邊的位集。O(1)



140位 點陣圖 的 優先事項 大批（積極的 位數組 & 已到期 位數組）

- 壞的：低的互動的效能（低優先權進程）



積極的大批

Linux進程調度器

- 在）前核心 2.4
- O(1) 核心 2.5
- 完全地公平的調度器（糧安委）核心 2.6 O(log(N))
 - 目標：更多互動等“公平的”比 O(1)。
 - 概念：（1）每個行程不再靜態映射時間片，而是接收額）。（2）以無限短的持續時間調度進程。

虛擬運行時間：真實系統運行時間+優先權

（保持所有正在運行的進程的虛擬運行時間相同）

分配的CPU時間= $_\text{sched_period}() * \text{NICE_Weight} / \text{NICE_k_Weight}$

（理論） $\text{Vruntime} += \text{delta_exec} * \text{NICE_0_Weight} / \text{NICE_Weight}$

（實用） $\text{Vruntime} += (\text{delta_exec} * \text{NICE_0_Weight} * 2^{32} / \text{NICE_Weight})$

$\gg 32$ $_\text{sched_period}$ ：週期 進入調度程序（例如，6ms或0.75ms（常數））

delta_exec ：進程的執行時間（或稱掛鐘時間）

NICE_Weight ：對應nice值的權重（例如，0 -> 1024）

NICE_k_Weight ：總體所有k個正在運行的進程的權重

例子

兩個進程：P1（Nice 0），P2（Nice 19）

總體 CPU：

P1： $6\text{ms} * 1024 / (1024 + 15) = 5.9\text{ms}$

P2： $6\text{ms} * 15 / (1024 + 15) = 0.1\text{ms}$

- 初始Vruntime為 0

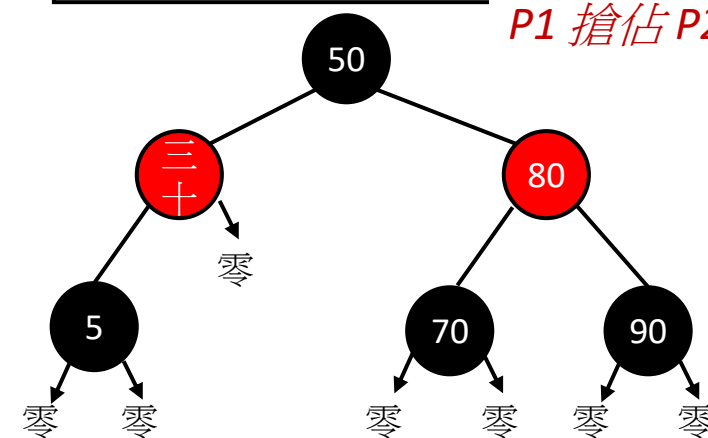
- P1先運行，1ms後面向I/O

- 0.05ms 後 P1 完成，P2 接管

P1的vruntime += $1 * 1024 / 1024 = 1$

P2的vruntime += $0.05 * 1024 / 15 = 3$

P1 搶佔 P2 !



小的 虛擬運行時（指數） 大的
最需要CPU 最少需要CPU

Linux進程調度器（數數。）

- 在）前核心 2.4
- O(1) 核心 2.5
- 完全地 公平的 調度器（糧安委）核心 2.6
 - 目標：更加 互動且 “公平” 比 O(1)。
 - 重量如何決定？

答案：遵循10%效應

nice level	priority	weight
-20 (min)	100	$88761 \approx 1024 \times 1.25^{20}$
$n \in [-19, -1]$	$120 + n$	$\approx 1024 \times 1.25^{-n}$
0 (default)	120	1024
$n \in [1, 18]$	$120 + n$	$\approx 1024 \times 1.25^{-n}$
19 (max)	139	$15 \approx 1024 \times 1.25^{-19}$

虛擬運行時

參考文獻1：<https://developer.ibm.com/tutorials/l-completely-fair-scheduler/>

參考2：<https://codingnote.cc/zh-tw/p/78397/>

Ref3：<https://mechpen.github.io/posts/2020-04-27-cfs-group/index.html> Ref4

：<https://hackmd.io/@RinHizakura/B18MhT00t>

$$\text{runtime} = \text{delta_exec} * \frac{\text{NICE_Weight}}{\text{NICE_k_Weight}}$$

```

/*
 * Nice levels are multiplicative, with a gentle 10% change for every
 * nice level changed. I.e. when a CPU-bound task goes from nice 0 to
 * nice 1, it will get ~10% less CPU time than another CPU-bound task
 * that remained on nice 0.
 *
 * The "10% effect" is relative and cumulative: from _any_ nice level,
 * if you go up 1 level, it's -10% CPU usage, if you go down 1 level
 * it's +10% CPU usage. (to achieve that we use a multiplier of 1.25.
 * If a task goes up by ~10% and another task goes down by ~10% then
 * the relative distance between them is ~25%.)
 */
static const int prio_to_weight[40] = {
    /* -20 */    88761,    71755,    56483,    46273,    36291,
    /* -15 */    29154,    23254,    18705,    14949,    11916,
    /* -10 */    9548,    7620,    6100,    4904,    3906,
    /* -5 */     3121,    2501,    1991,    1586,    1277,
    /* 0 */      1024,    820,    655,    526,    423,
    /* 5 */       335,    272,    215,    172,    137,
    /* 10 */      110,    87,    70,    56,    45,
    /* 15 */       36,    29,    23,    18,    15,
};

```

Linux 核心 4.4

(18-15) / (18+15) = ~10%

即時調度策略

- 柔軟的 即時的：核心嘗試在時間期限內調度應用程式。（Linux 僅有的提供 柔軟的 即時的）
- 難的 即時的：系統保證在一定限度內滿足任何調度要求。
- Linux 即時的 調度策略：
 - 優先事項 範圍：0~99（即時進程），100~139（正常進程）
 - 先進先出（**SCHED_FIFO**）：任務繼續到跑步直到它區塊或者明確地殖利率這處理器。先進先出任務有不時間片和能跑步無限期地。
 - 僅有的一個高的優先事項先進先出或者RR任務能先發制人。
 - 圓形的羅賓（**SCHED_RR**）：先進先出使用時間片。
進程永遠無法搶先**SCHED_RR**任務，即使其時間片已用盡。

行程上下文切換

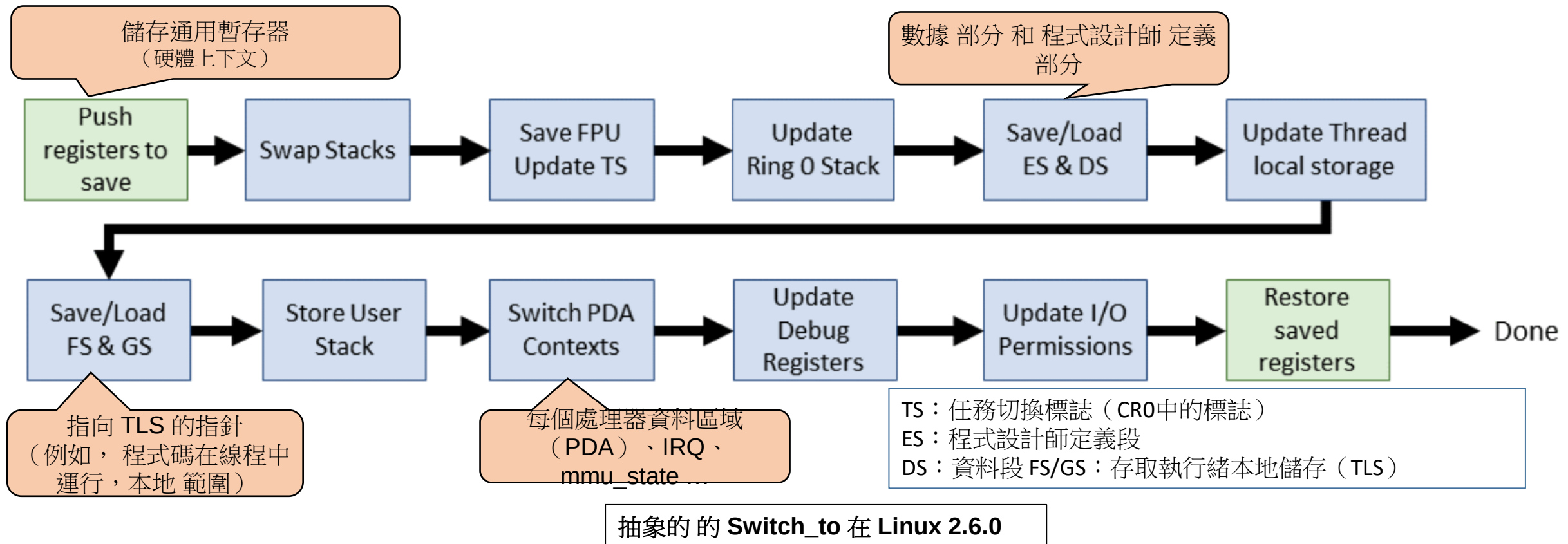
- 過程 上下文：使用者地址空間的內容（例如，頁 行程資訊包括行程表（即行程狀態表）、硬體暫存器（如PC）以及與行程相關的**核心資料結構**。
 - 你也可以這樣想，上下文非常類似運行某些東西的「環境」。
- 行程上下文 轉變：交換 從 一 可運行 過程 到 其他。
 - `switch_mm()`：轉變 這 虛擬的 記憶 對應（例如，頁 桌子，薄層負載平衡器（TLB）



抽象的的 開關_mm () 在 Linux 4.4

進程上下文切換（計數）

- 行程上下文交換：交換從一可運行過程到其他。
 - `switch_mm()`：轉變這虛擬的記憶對應（例如，頁桌子，TLB，快取）
 - 切換到（）：轉變這過程狀態（例如，處理器登記，特定於體系結構狀態）



CPU 模式切換 vs 進程上下文切換

- 當遇到異常或中斷時，CPU從使用者模式切換到核心模式，運行核心程式碼。
 - CPU 也應該進行上下文切換，但**不能進行進程級上下文切換**
- 在切換到核心模式之前，CPU應該會**保存用戶模式上下文**
 - 使用者模式上下文：程式計數器、使用者模式特定狀態（例如，使用者空間堆疊指標暫存器（ESP或RSP）、浮點暫存器、使用者空間指令指標）
 - 注意：當我們說保存上下文時，我們通常意味著將**所有這些暫存器寫回主記憶體中**



系統調用

什麼是系統調用

- 目標：讓使用者空間 流程與操作交互 **system**，核心提供了一組接口，稱為系統呼叫。
- 運作方式：系統呼叫是訪問透過 **功能呼叫** 定義在這碳圖書館。



監視器是一個
字元設備！
(有文件支援)

- 系統 稱呼 編號：每個系統呼叫都有一個**唯一的編號**。
(運行時無法更改所有對應的編號。)
- `sys_call_table`：核心在系統呼叫表中

```
# The format is:
# <number> <abi> <name> <entry point>
#
# The __x64_sys_*() stubs are created on-the-fly for sys_*() system calls
#
# The abi is "common", "64" or "x32" for this file.
#
0      common  read      __x64_sys_read
1      common  write     __x64_sys_write
```

[Linux 4.17.4](#) → [/arch/x86/entry/系統呼叫/syscall_64.tbl](#)

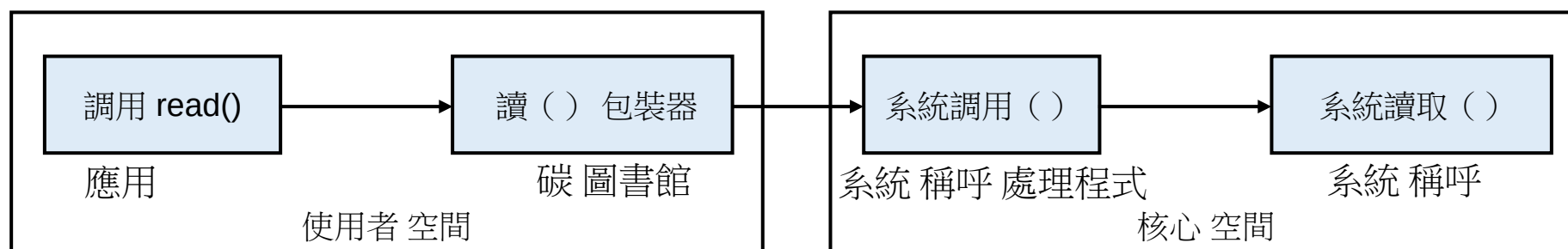
系統 稱呼 處理程式

- 如何到跑步系統稱呼：
 - 使用者空間應用程式必須不知何故 **訊號** 到這核心 → 系統稱呼處理程式
 - 這中央處理器開關從 **使用者模式** 到 **核心模式**（不是上下文切換！！）
 - 執行程式碼相應的到這系統稱呼
- 系統詳情稱呼處理程式
 - 如何到訊號 → 軟體打斷（IE，一個例外）
 - 跑步 INT \$0x80 → 打斷操作說明，數位 128（運行這系統稱呼處理程序）



如何實現 系統調用

- 添加（註冊）一個新的系統稱呼到 Linux 是簡單的 和 做不是需要到是擔心的 和 這行為的這系統稱呼處理程式



- 簡短的程式（細節：看參考）：
 - 寫一個新的系統稱呼（例如，`sys_hello.c`）
 - 創造一個 **Makefile**
 - 添加這新的系統稱呼進入這系統稱呼桌子
 - 參數：系統調用號，拱門（例如，**64**），系統呼叫名稱，系統呼叫程式
 - 添加這新的系統稱呼到這標題文件（包括 `/linux/syscalls.h`）
 - 編譯/安裝/更新 核心

[1] <https://medium.com/anubhav-shrimal/adding-a-hello-world-system-call-to-linux-kernel-dad32875872>。

[2] <https://tssurya.wordpress.com/2014/08/19/adding-a-hello-world-system-call-to-linux-kernel-3-16-0/>

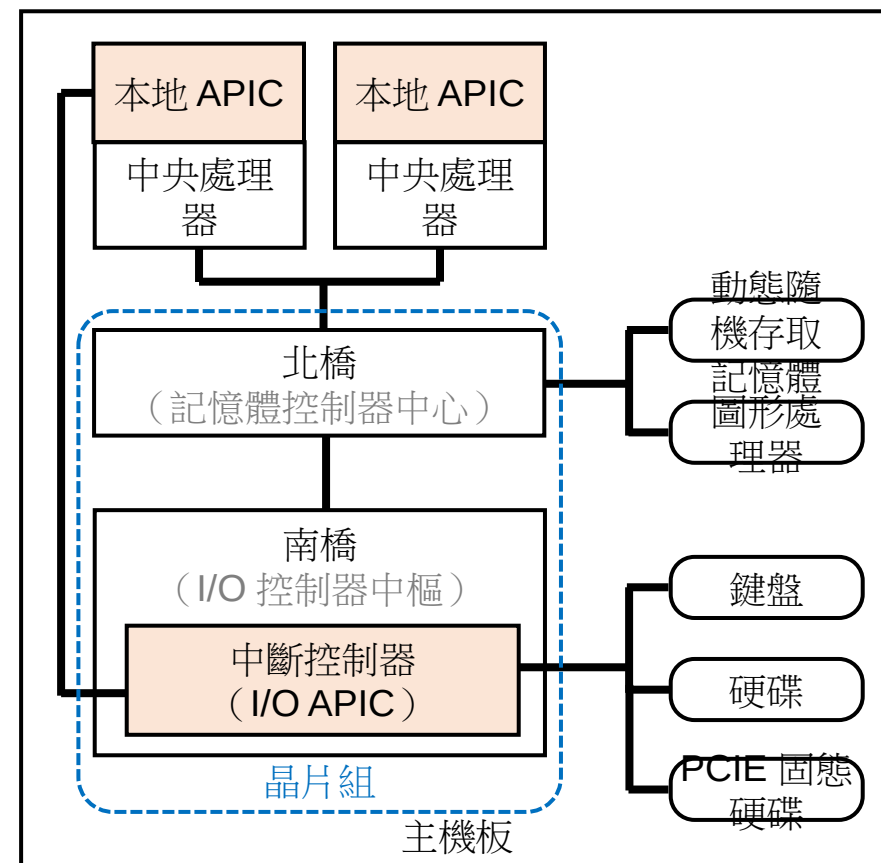
[3] <https://dev.to/jasper/adding-a-system-call-to-the-linux-kernel-5-8-1-in-ubuntu-20-04-lts-2ga8>



中斷

處理硬體事件

- CPU 如何與較慢的硬體設備協同工作
 - 輪詢：核心定期**檢查硬體**狀態並做出相應回應。
 - 中斷：讓硬體向核心
- 導致中斷的原因：
 - 中斷（非同步，硬體中斷）：硬體設備**物理產生電子訊號**，這些訊號直接送入**中斷控制器的輸入引腳**。



注意：2019 年後，所有北橋功能已經整合到CPU中了！！

處理硬體事件（計數）

- CPU 如何與較慢的硬體設備協同工作
 - 輪詢：核心定期檢查硬體狀態並做出相應回應。
 - 中斷：讓硬體向核心
- 導致中斷的原因：
 - 中斷（非同步、硬體中斷）：硬體設備物理產生電子訊號，這些訊號直接送入中斷控制器的輸入引腳（例如，鍵盤，磁碟）。
 - 異常（同步、軟體中斷）：異常是由處理器在執行指令時產生的，或是為了回應程式錯誤（例如，劃分經過零）或異常狀態（例如，稱呼一個系統稱呼或者一個頁過錯）。

中斷請求（IRQ）

中斷向量號（IVN）

IVN	Default processor IVT	Remapped by BIOS	Remapped by an OS Honoring the default IVT
0x00h	Divide by zero	Divide by zero	Internal interrupts
0x01h	Single step trace	Single step trace	
0x02h	NMI, power failure	NMI, power failure	
0x03h	Breakpoint	Breakpoint	
0x04h	Numeric overflow	Numeric overflow	
0x05h	Screen dump to printer	Screen dump to printer	
0x06h	Invalid instruction	Invalid instruction	
0x07h	No coprocessor	No coprocessor	
0x08h	Double fault	IRQ0: system timer	
0x09h	Coprocessor segment overrun	IRQ1: keyboard	
0x0Ah	Invalid task state segment (TSS)	IRQ2: cascade	
0x0Bh	Segment not present	IRQ3: Serial COM2/4	
0x0Ch	Stack segment overrun	IRQ4: Serial COM1/3	
0x0Dh	General protection fault (GPF)	IRQ5: Sound card	
0x0Eh	Page fault	IRQ6: floppy disk ctr	
0x0Fh	Reserved	IRQ7: Parallel port	
0x10h	Coprocessor error	BIOS: video services	Reserved
0x11h	Alignment check (486+only)	BIOS: check	
0x12h	Machine check (Pentium+only)	BIOS: memory size	
0x13h	Reserved	BIOS: disk services	
0x14h		BIOS: serial ports	
0x15h		BIOS: system services	
0x16h		BIOS: keyboard services	
0x17h		BIOS: Printer services	
0x18h		BIOS: BASIC	
0x19h		BIOS: load OS	
0x1Ah	BIOS: other services		
0x1Fh			
0x20h			
0x21h	DOS services	Software interrupts	
0x70h	IRQ8 : real-time clock		
0x71h	IRQ9 : PCI device		
0x72h	IRQ10: Reserved		
0x73h	IRQ11: Reserved		
0x74h	IRQ12: PS/2	OS Services	
0x75h	IRQ13: 8087		
0x76h	IRQ14: Hard Disk 1		
0x77h	IRQ15: Hard Disk 2		
0x80h		Software interrupts	Master PIC
0x88h			Slave PIC
0xFFh		Software interrupts	Software interrupts

X86中斷向量表
(拱門依賴)

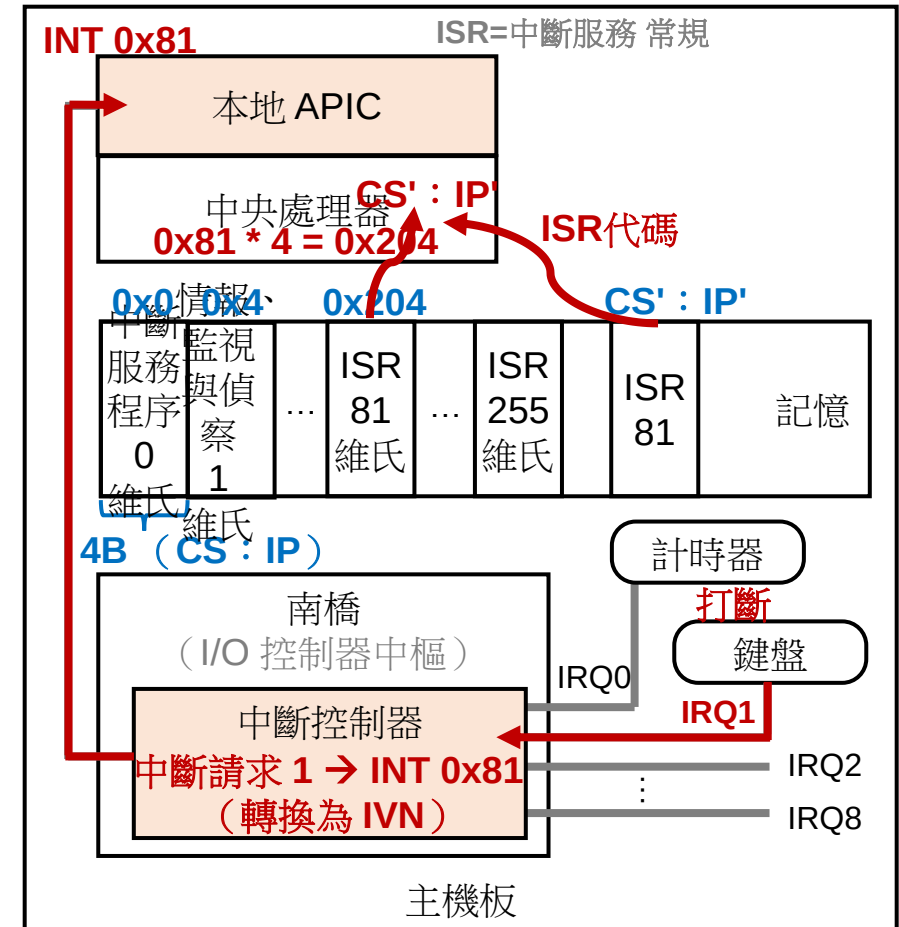
X86中斷向量表

（拱門依賴）

- 不同的設備可以透過與每個中斷關聯的**唯一中斷值（最大值：256）**與不同的中斷相關聯（就像系統呼叫一樣！）
- 中斷請求(IRQ)：
 - 特定的中斷與特定的設備相關聯，內核知道這一點。（啟動時分配）
 - IRQ 0 → 定時器中斷
 - IRQ 1 → 鍵盤中斷
 - 動態分配 IRQ # → PCI 設備

中斷如何運作

- 中斷的工作原理：
 - 硬體設備發出中斷訊號。
 - 中央處理器 停止當前執行。
 - 中斷控制器發送INT # 到 CPU。
 - 中央處理器 訪問 這打斷向量桌子 經過 $INT \# * 4$
 - CPU 運作 打斷服務 處理中斷的



中斷處理程序

- 打斷服務例行程序（ISR）或者打斷處理程序：這功能這核心運行在回覆到一個具體的打斷。
 - 它是部分的一個裝置司機（這核心程式碼管理這裝置）
- 挑戰：ISR不可阻塞（無法休眠）。處理程序應快速運行，以便盡快恢復中斷程式碼的執行。
- 將ISR分為兩部分（上半部 + 下半部）：
 - 上半部：在收到中斷後立即運行，並且僅執行時間關鍵的工作。
 - 下半部：可以稍後執行的工作將延後到下半部。
 - 例子：
 1. 網路卡接收到一個資料包。
 2. 上半部將封包從網路卡中的緩衝區移到主機記憶體。
 3. Bottom halves 將稍後處理該資料包。